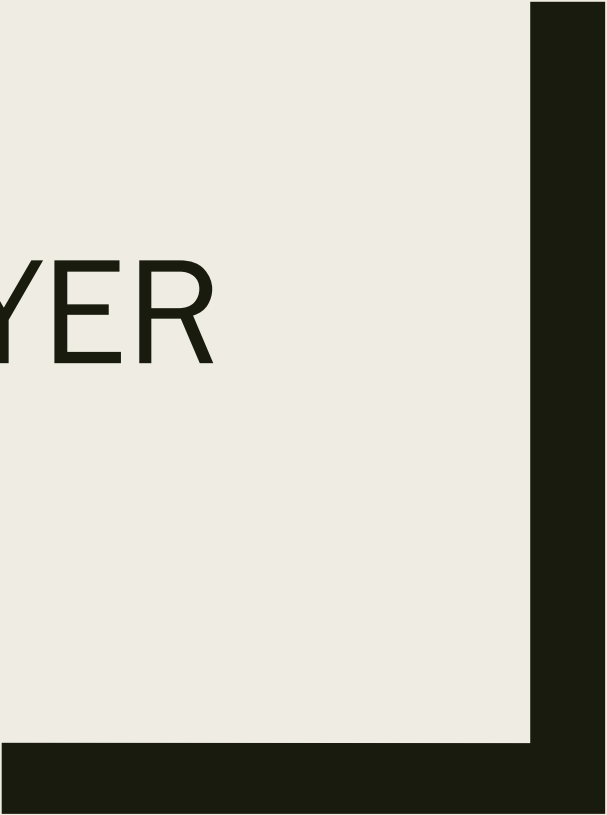


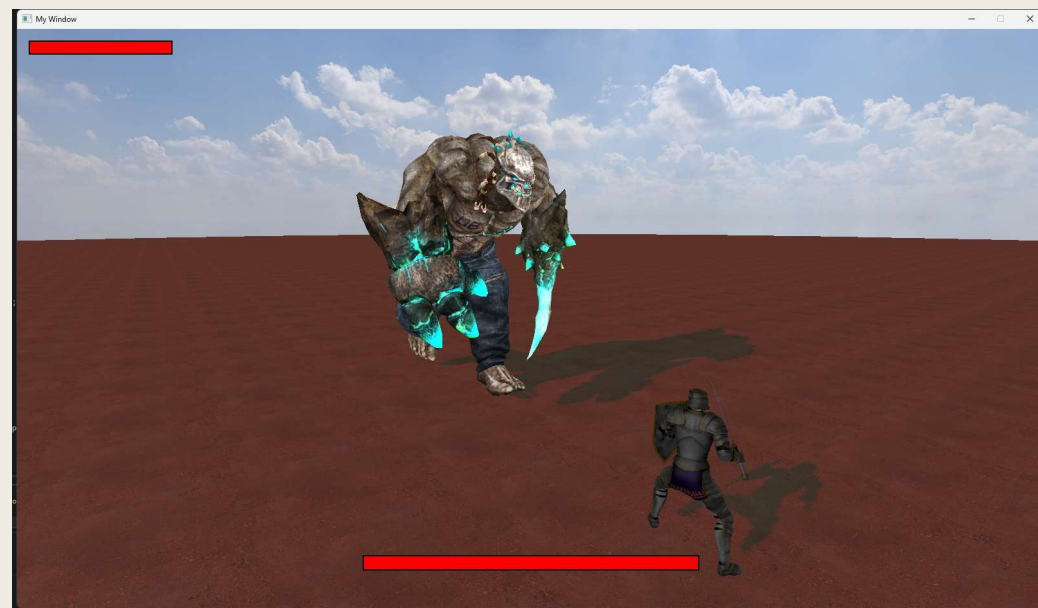


MONSTER SLAYER

CHEN RUOYU

操作説明

- W/A/S/D: 移動
- マウス移動: 視点操作
- マウス中ボタン: 敵をロックオン(推奨)
- 左クリック: 攻撃
- 右クリック: パリイ



特殊スキル：

パリィ成功時に黄色い閃光が見えたら、攻撃ボタンを押すことで強力な一撃を繰り出せます。

ぜひ防御反撃を積極的に活用してください。



コア モジュール

•FBXアニメーションの読み込み／描画パイプライン

- Assimp を用いた自作インポート処理により、FBX アニメーション資産を .anim / .skel / .mat / .mesh に分解・変換
- .anim から各フレイbones poseを算出
- .skel の階層情報に基づき、pose をスキニング用Bone Matrixへ変換
- Bone Matrix をGPUへアップロードし、Vertex Shader が .mesh をスキニング変形
- .mat でテクスチャをバインドし、最終描画を実行

sword_block_attack.fbx

sword_block.anim

sword_block.mat

sword_block.mesh

sword_block.skel

```
{
  AnimClipDesc c{};
  c.name = L"Block";
  c.meshPath = L"resources/player_anim/cooked/sword_block.mesh";
  c.skelPath = L"resources/player_anim/cooked/sword_block.skel";
  c.animPath = L"resources/player_anim/cooked/sword_block.anim";
  c.matPath = L"resources/player_anim/cooked/sword_block.mat";
}
```



コア モジュール

•FBXアニメーションの読み込み／描画パイプライン

•アニメーション読み込み時に Root Motion の適用方式を選択可能 (None / UseVelocity / UseDeltaZ / UseAnimDelta)

•選択した方式に応じて、アニメ由来の移動成分を異なるルールで処理 (全削除／速度駆動／Z成分のみ保持／XZ成分保持)

•ゲーム内の設計意図に合わせて、状況ごとに適切な設定を使い分け可能

```
enum class RootMotionType : uint8_t {
    None = 0, // 不使用动画位移
    VelocityDriven = 1, // 由速度驱动, 动画只摆动
    UseAnimDelta = 2, // 使用动画的 Δ 位移/Δ 朝向
    UseZDelta = 3, // 只使用本地 Z 轴位移 (前进距离), 忽略 ΔYaw
};
```

```
AnimClipDesc c();
c.name = L"Parry";
c.meshPath = L"resources/player_anim/cooked/sword_parry.mesh";
c.skelPath = L"resources/player_anim/cooked/sword_parry.skel";
c.animPath = L"resources/player_anim/cooked/sword_parry.anim";
c.matPath = L"resources/player_anim/cooked/sword_parry.mat";
c.baseColorOverride = L"resources/player_anim/cooked/Textures/Paladin_diffuse.png";
c.loop = false;
c.playbackRate = 1.0f;
c.rmType = RootMotionType::UseZDelta;
```

```
switch (clip.rmType)
{
    case RootMotionType::None:
        zeroXZ = true;
        break;

    case RootMotionType::VelocityDriven:
        // 这些模式完全不用动画位移 → 必须清除根平移
        zeroXZ = true;
        break;

    case RootMotionType::UseZDelta:
        // Roll: 只靠 Z 位移做 RootMotion, 视觉上不希望骨骼自己再走一圈
        zeroXZ = true;
        break;

    case RootMotionType::UseAnimDelta:
        // 完整 RootMotion: 可以先保留骨骼平移 (如果以后觉得是 double, 可以再一起改)
        zeroXZ = false;
        break;
}

// 1) 清 motion-root 的 XZ (你原本就有)
ModelSkinned_SetZeroRootTranslationXZ(zeroXZ);

// 2) 再额外清 Hips/Pelvis 的 XZ (关键)
if (zeroXZ) {
    ModelSkinned_SetZeroTranslationXZBoneByName("hips");
}
else {
    ModelSkinned_SetZeroTranslationXZBoneByName(nullptr);
}
```

コア モジュール

- FSM／遷移時のアニメーションブレンド／FSMのJSON設定・読み込み

- FSM用JSON設定と対応する読み込みツールを整備し、全体をデータ駆動(Data-driven)化することで、変更・調整・デバッグを容易化

- Stateの遷移(IN/OUT)タイミング情報をアニメ描画側へ連携し、アニメーション側でブレンド処理を実行

- State遷移は、遷移元／遷移先(IN/OUT State)と遷移条件によって駆動

- ゲームロジック側は、FSMの遷移条件(フラグ／パラメータ等)の更新・トリガのみを担当

```
{
  "name": "Block",
  "clip": "Block",
  "loop": false,
  "root_motion": "use_delta",
  "length_sec": 0.0,
  "locomotion": false,
  "uninterruptible": [ [ 0.0, 0.3 ] ]
},
{
  "name": "Block_Attack",
  "clip": "Block_Attack",
  "loop": false,
  "root_motion": "use_delta",
  "length_sec": 0.0,
  "locomotion": false,
  "uninterruptible": [ [ 0.0, 0.35 ] ]
},
}
```

```
{
  "from": "Block",
  "to": "Block_Attack",
  "trigger": "Attack",
  "buffer": 0.15,
  "window": [ [ 0.00, 1.00 ] ],
  "duration": 0.05,
  "curve": "linear",
  "can_interrupt": true,
  "priority": 150
},
}
```

```
PlayerSM_SetMoveInput(in.moveX, in.moveZ);
if (in.attack) {
  PlayerSM_FireTrigger("Attack");
}
```



コア モジュール

- フレームイベントシステム (攻撃判定 / VFX再生 / カメラ切替 など)

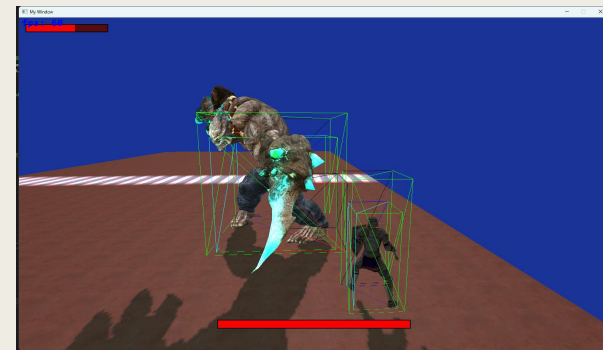
- キャラクターアニメーションをフレームイベントで駆動し、各種ゲームインタラクションを実行 (設定・管理が明確で追跡しやすい)

- 各アニメーションを1つのTrackとして扱い、正規化時間に基づいてTrack内のフレームイベントを順次実行

- 攻撃判定を共通基盤化し、被弾処理の統一入口 (共通API) を設けて後続の拡張・管理を容易化

- フレームイベント用の専用設定コード / ツールを用意し、保守性と運用性を確保

```
{  
    AnimEvent e{};  
    e.timeNormalized = 0.0f;  
    e.type = AnimEventType::SetHurtBoxEnabled;  
    e.setHurtBox.enabled = false;  
    roll.events.push_back(e);  
}  
  
{  
    AnimEvent e{};  
    e.timeNormalized = 0.85f;  
    e.type = AnimEventType::SetHurtBoxEnabled;  
    e.setHurtBox.enabled = true;  
    roll.events.push_back(e);  
}  
AnimEventRegistry_Register(roll);  
}
```



```
if (c.attackerOwner == Boss_GetHitboxOwnerToken() &&  
    c.victimOwner == Player_GetHurtboxOwnerToken())  
{  
    HitParams hp{};  
    hp.attackerOwner = c.attackerOwner;  
    hp.victimOwner = c.victimOwner;  
    hp.damage = c.damage;  
    hp.level = ChooseHitLevelFromDamage(c.damage);  
    hp.hitPos = c.hitPos;  
  
    hp.knockbackDistance = c.knockbackDistance;  
    hp.attackerPos = c.attackerPos;  
    hp.victimPos = c.victimPos;  
  
    PlayerHitResponse r = Player_OnIncomingHit(hp);  
  
    char buf[256];  
    switch (r)  
    {  
    case PlayerHitResponse::Ignored:  
        sprintf_s(buf, "[HitEvent] BOSS hit PLAYER -> IGNORED (invincible) dmg=%d\n", c.damage);  
        OutputDebugStringA(buf);  
        return true; // * 被弾消費 HitBox (按你补充规则)  
    case PlayerHitResponse::Parried:  
        sprintf_s(buf, "[HitEvent] BOSS hit PLAYER -> PARRIED dmg=%d\n", c.damage);  
        OutputDebugStringA(buf);  
        return true; // * 消耗 HitBox  
    case PlayerHitResponse::TookHit:  
    default:  
        sprintf_s(buf, "[HitEvent] BOSS hit PLAYER! dmg=%d\n", c.damage);  
        OutputDebugStringA(buf);  
        return true; // * 消耗 HitBox  
    }  
}
```


コア モジュール

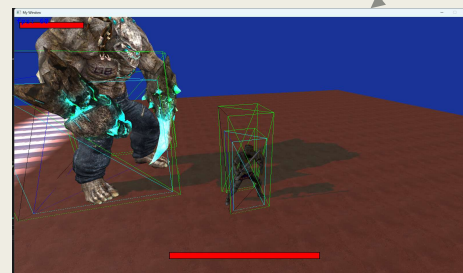
- 複数のカメラ表現(フリー／ロックオン／スキル演出カメラ)

- フリーカメラ／ロックオンカメラの2モードを実装し、モードに応じてプレイヤー移動の入力軸(基準座標)を切り替え

- スキル演出用の専用カメラを実装し、アニメーション中のフレームイベントから呼び出すことで、アクションゲームらしい運鏡演出を実現

- カメラシェイクを実装(方向／強度／時間を調整可能)。フレームイベントから制御でき、スキルカメラとも干渉しない構成に整理

```
enum class PlayerCameraMode {  
    Free,  
    LockOn,  
};
```



```
// 1) LockOn preset  
{  
    AnimEvent e{};  
    e.timeNormalized = 0.0f;  
    e.type = AnimEventType::CameraLockOnPreset;  
    e.cameraLockOnPreset.presetId = 1; // 1 = Block_Attack  
    e.cameraLockOnPreset.blendInSec = 0.2f;  
    e.cameraLockOnPreset.holdSec = 1.12f;  
    e.cameraLockOnPreset.blendOutSec = 0.25f;  
    e.cameraLockOnPreset.priority = 200;  
    t.events.push_back(e);  
}  
  
// 2) Camera shake  
{  
    AnimEvent e{};  
    e.timeNormalized = 0.5f;  
    e.type = AnimEventType::CameraShake;  
    e.cameraShake.magnitude = 0.58f;  
    e.cameraShake.durationSec = 1.12f;  
    e.cameraShake.mode = CameraShakeMode::Both;  
    e.cameraShake.priority = 100;  
    t.events.push_back(e);  
}
```